# OASIS: Supplementary Materials

Author Names Redacted for Review

Affiliation Redacted
redacted@review.org

This document collects supplementary materials for the paper "OASIS: Feedback-Driven Statistics Correction for Cost-Based Query Optimizers." It provides detailed scope and error-budget positioning (§ A), model architecture and instrumentation details (§ B), baseline fairness discussion and QuickSel-H adaptation (§ C), the portable statistics-format conversion interface with PostgreSQL MCV instantiation (§ D), predicate-coverage sensitivity analysis (§ E), observation aggregation study details (§ F), optimizer-only EXPLAIN protocol details (§ G), and detailed overhead breakdowns (§ H).

## 1 Scope and Error Budget

OASIS targets *single-column histogram staleness*. We explicitly exclude from scope:

- **Multi-column correlation errors**: Join cardinality misestimation caused by correlated column predicates requires multi-dimensional statistics (e.g., mvdists, DNF-based sketches, or learned joint models). OASIS operates on one-dimensional marginals and is complementary to multi-column methods.
- **Physical design errors**: Missing indexes, suboptimal partitioning, or incorrect materialized views are outside the statistics layer.
- **Cost-model calibration**: Even with correct selectivities, the optimizer's cost model may mispredict operator runtimes. OASIS improves selectivity estimates but does not modify the cost model.
- **Data distribution shifts beyond drift**: Fundamental schema changes (column additions, type changes) or massive bulk loads that alter the distribution shape beyond what a compound drift operator captures are out of scope; these scenarios require a fresh `ANALYZE`.

*Error budget decomposition.* In a production optimizer, the total cardinality estimation error decomposes as:

$$\text{Total Error} = \underbrace{\text{Single-Column Error}}_{\text{OASIS target}} + \underbrace{\text{Correlation Error}}_{\text{multi-column methods}} + \underbrace{\text{Cost-Model Error}}_{\text{calibration}} + \underbrace{\text{Physical Design Error}}_{\text{indexing/partitioning}}.$$

OASIS addresses the first term. Even perfect single-column statistics do not eliminate the remaining terms, but accurate marginals are a necessary prerequisite: learned multi-column estimators (e.g., copula-based models like CoLSE [4]) combine one-dimensional marginals with a learned dependence structure, making single-column statistics quality a foundational concern.

## 2 Model and Instrumentation Details

This appendix collects implementation details trimmed from the main paper for page budget reasons.

### 2.1 Attention-Pooled MLP Architecture

The OASIS correction model is an *attention-pooled MLP* that exploits non-linear interactions between the prior histogram and the observation window. The architecture is deliberately lightweight for CPU deployment:

1. **Prior encoder**: The stale prior quantiles $\mathbf{v}_{\text{norm}}$ are first processed through a dedicated single-layer MLP ($9 \to 32 \xrightarrow{\text{ReLU}} 32$) to extract high-level distributional features before fusion with observations.
2. **Multi-head attention**: Each observation slot $o_i \in \mathbb{R}^{D_{\text{obs}}}$ receives $H{=}3$ independent attention scores $a_i^{(h)} = \mathbf{w}_{\text{attn}}^{(h)\top} o_i + b_{\text{attn}}^{(h)}$, one per head $h \in \{1, 2, 3\}$. Each head's scores are masked (padded slots zeroed) and passed through softmax, yielding $H$ attention weight vectors $\boldsymbol{\alpha}^{(h)} \in \Delta^{K-1}$.
3. **Multi-head pooling**: Each head produces a pooled observation vector $\mathbf{p}^{(h)} = \sum_i \alpha_i^{(h)} o_i \in \mathbb{R}^{D_{\text{obs}}}$. The $H$ pooled vectors are concatenated to form a multi-perspective observation summary $\mathbf{p}_{\text{all}} = [\mathbf{p}^{(1)} \,||\, \cdots \,||\, \mathbf{p}^{(H)}]$.
4. **Deep residual MLP**: The context vector $\mathbf{c} = [\mathbf{v}_{\text{enc}} \,||\, \mathbf{m} \,||\, \mathbf{p}_{\text{all}}]$ (encoded prior $\|$ meta $\|$ multi-head pooled observations) is fed into a four-hidden-layer MLP with skip connections: $\mathbf{c} \to 128 \xrightarrow{\text{ReLU}} 128 \xrightarrow{\text{ReLU}} 64 \xrightarrow{\text{ReLU}} 64 \xrightarrow{\text{ReLU}} (B{-}1)$. Skip connections are added every two layers to facilitate gradient flow.
5. **Residual prediction**: The MLP output $\boldsymbol{\delta}$ represents a *correction delta* rather than absolute quantile values. The final corrected histogram is computed as $\mathbf{v}_{\text{corrected}} = \mathbf{v}_{\text{norm}} + \boldsymbol{\delta}$, allowing the model to focus on learning drift-induced deviations.

*Training details.* The OASIS MLP is trained offline on simulation-derived ground truth. A *compound drift operator* mutates an in-memory column through four atomic operations per round—**insert**, **delete**, **update**, and **null change**—with drift intensity $q$ controlling the number of rounds between consecutive query observations. For each simulated sample the simulator records the true post-mutation quantile vector $\mathbf{v}^*$ as the regression target. The model is trained end-to-end via mini-batch Adam with He weight initialization, L2 regularization $\alpha{=}10^{-4}$, and gradient clipping (max norm = 1.0). The complete implementation is $\sim$550 lines of pure Python + NumPy (no external ML framework), totalling $\approx$38 K parameters.

### 2.2 Per-Conjunct Operator Instrumentation

Filter and scan-filter-project operators are augmented with per-conjunct selectivity counters. During execution, whenever a batch of rows is processed through a

compound predicate $\phi_1 \wedge \phi_2 \wedge \cdots \wedge \phi_m$, each conjunct $\phi_j$ is evaluated independently on the full input batch:

$$n_j^{\text{in}} \mathrel{+}= |\text{batch}|, \qquad n_j^{\text{out}} \mathrel{+}= |\{r \in \text{batch} : \phi_j(r)\}|.$$

The final intersection of all conjunct results produces the same output as the original compound filter while additionally recording per-conjunct independent selectivities $s_j^* = n_j^{\text{out}}/n_j^{\text{in}}$. Upon operator completion, these counters are written to the engine's existing runtime metrics store. In the deployment model used by OASIS, the collector simply piggybacks on predicate evaluation already being performed by the engine, flushes a structured record to disk, and emits at most one observation tuple per filter conjunct. As a result, each query writes at most as many observation records as it has conjunctive filter clauses.

**Algorithm 1** Per-Conjunct Operator Instrumentation

**Require:** Compound filter $\Phi = \phi_1 \wedge \cdots \wedge \phi_m$, input batch $B$
**Ensure:** Filtered output $B'$, counters $n_j^{\text{in}}, n_j^{\text{out}}$

mask $\leftarrow$ [**true**]$^{|B|}$
**for** $j = 1$ **to** $m$ **do**
    result$_j \leftarrow \phi_j(B)$
    $n_j^{\text{in}} \mathrel{+}= |B|$;  $n_j^{\text{out}} \mathrel{+}= |\text{result}_j|$
    mask $\leftarrow$ mask $\wedge$ result$_j$
**end for**
**return** $B$[mask]

*// On operator completion, emit per-conjunct metrics:*
**for** $j = 1$ **to** $m$ **do**
    EmitMetric(conjunct_$j$, $n_j^{\text{in}}$, $n_j^{\text{out}}$)
**end for**

### 2.3 QuickSel-H Adaptation Details

The QuickSel system [3] is a selectivity learner that fits a mixture-model density to satisfy feedback constraints. Because QuickSel outputs point selectivity estimates rather than histogram boundaries, we evaluate a clearly labeled **QuickSel-H** adaptation that bridges the gap between selectivity learning and histogram correction. The adaptation performs three steps:

1. **Mixture-model fitting**: Given the same stale prior $H_0$ and observation window $\mathcal{O}$ of size $K$=16, QuickSel-H fits a Gaussian mixture model with $M$=5 components whose CDF is constrained to match observed selectivities at the predicate values in $\mathcal{O}$. The optimization uses the same relative-entropy objective as the original QuickSel, solved via iterated scaling.
2. **Quantile extraction**: From the fitted mixture CDF, we extract the $B-1$=9 quantile values at the equi-depth levels used by all other methods. This ensures QuickSel-H outputs on the same histogram grid.
3. **Output interface**: The extracted quantiles are passed through the same monotonic projection as OASIS and returned as the corrected histogram.

**Why this adaptation is necessary.** QuickSel's native output is a density estimate over the full domain, not a fixed-resolution histogram consumable by the CBO. To compare under a matched output interface, we must extract quantiles from this density. The extraction step introduces a minor information bottleneck (compressing a rich density into $B-1$ boundaries), but this is identical to the bottleneck all other methods face and does not fundamentally change QuickSel's correction mechanism. We do not claim QuickSel-H is a line-by-line reproduction of the original system; it is a faithful adaptation that preserves QuickSel's core density-fitting approach while conforming to the common output format required by our evaluation protocol.

## 3 Baseline Fairness Discussion

All feedback-driven methods compared in the main paper consume the same observation window of size $K$=16 and emit corrected quantiles on the same $B$=10 equi-depth histogram grid. No method receives additional information or computational budget beyond this shared interface.

*STHoles.* We implement the hierarchical refinement algorithm described in [1]. STHoles builds a tree of bucket refinements from feedback. We initialize it from the stale prior and allow up to $K$ refinement steps per correction instance, using the same observation tuples provided to OASIS. The output quantiles are extracted from the refined tree on the standard $B-1$ grid.

*ISOMER.* We implement the consistency-based histogram adjustment from [2]. ISOMER adjusts bucket frequencies to satisfy observed selectivity constraints while minimizing KL divergence from the prior. We provide the same $K$=16 observation tuples and extract quantiles on the standard grid.

*QuickSel-H.* See § 2 for the detailed QuickSel-H adaptation.

*Simple heuristics.* LinInterp and FeedAvg consume the same observation window but use fixed aggregation rules (blend factor 0.5 and exponential moving average, respectively). They are included to demonstrate that learned correction is necessary, not merely that any feedback-driven approach helps.

No method receives per-table or per-distribution tuning; all use default hyperparameters. Results are averaged over three independent training seeds (42, 123, 456) with 95% confidence intervals.

## 4 Portable Statistics-Format Conversion Interfaces

### 4.1 Motivation

OASIS corrects a canonical full-distribution view of a column. This is straightforward when a DBMS exposes a self-contained histogram, but many production

systems do not. Instead, the optimizer's statistics object often couples the histogram with auxiliary summaries whose probability mass must remain consistent after any update.

- **PostgreSQL**: Maintains a Most Common Values (MCV) list alongside a residual histogram that excludes MCV entries.
- **Oracle**: Uses top-frequency values together with height-balanced histograms.
- **SQL Server**: Employs a density vector together with histogram steps.

This coupling is the main portability obstacle for histogram-correction methods: *correcting the histogram alone* is often insufficient because the stored statistics object is not a standalone histogram. OASIS addresses this with a *statistics-format conversion interface* that translates engine-specific statistics layouts into a canonical full distribution, applies OASIS there, and translates the corrected result back.

We instantiate the interface on PostgreSQL's MCV case because it is both practically important and representative of tightly coupled statistics architectures.

### 4.2 Three-Phase Conversion Interface

The interface follows three phases:

1. **Reconstruction**: Convert engine-native statistics → canonical full distribution
2. **Correction**: Apply OASIS on the canonical representation
3. **Decomposition**: Convert corrected full distribution → engine-native statistics

This design decouples OASIS's correction logic from database-specific storage formats. PostgreSQL is the running example in this appendix, but the pattern is more general.

### 4.3 PostgreSQL Instantiation

### 4.4 Problem Formulation

Given PostgreSQL statistics:

- MCV list: $\mathcal{M} = \{(v_1, f_1), (v_2, f_2), \ldots, (v_k, f_k)\}$
- Residual histogram bounds: $\mathcal{H} = \{u_0, u_1, \ldots, u_m\}$ where adjacent values define $m$ equi-depth residual intervals
- Null fraction: $f_{\text{null}}$
- Non-null invariant: $\sum_{i=1}^{k} f_i + p_{\text{res}} = 1 - f_{\text{null}}$

Construct a canonical full distribution $\mathcal{F}$ that exposes the complete non-null column distribution to OASIS.

### 4.5 Phase 1: Reconstruction

**Algorithm 2** Reconstruct Canonical Full Distribution from PostgreSQL Statistics

MCV list $\mathcal{M}$, residual bounds $\mathcal{H} = \{u_0, \ldots, u_m\}$, null fraction $f_{\text{null}}$ Canonical full distribution $\mathcal{F} = (\mathcal{S}, \mathcal{R})$ Initialize $\mathcal{S} \leftarrow \emptyset$, $\mathcal{R} \leftarrow \emptyset$ $p_{\text{res}} \leftarrow 1 - f_{\text{null}} - \sum_{(v_i, f_i) \in \mathcal{M}} f_i$ each $(v_i, f_i) \in \mathcal{M}$ Add singleton bucket $(v_i, f_i)$ to $\mathcal{S}$ $j = 1$ **to** $m$ Add range bucket $(u_{j-1}, u_j, p_{\text{res}}/m)$ to $\mathcal{R}$ Validate: $\sum_{s \in \mathcal{S}} s.\text{freq} + \sum_{r \in \mathcal{R}} r.\text{freq} = 1 - f_{\text{null}}$ $\mathcal{F} = (\mathcal{S}, \mathcal{R})$

PostgreSQL stores histogram *boundary values*, not explicit per-bucket frequencies. The conversion interface therefore interprets the residual histogram as $m$ equal-depth intervals carrying equal shares of the residual non-MCV mass. Before tensorization, the interface evaluates the CDF of $\mathcal{F}$ at OASIS's fixed quantile levels, yielding the canonical quantile vector consumed by the model.

### 4.6 Phase 2: OASIS Correction

Once reconstructed, the canonical full distribution is passed unchanged to the standard OASIS correction pipeline. The important systems property is that the model itself remains ignorant of the source database format; all engine-specific handling is localized to the conversion interface.

### 4.7 Phase 3: Decomposition

Given a corrected full distribution $\mathcal{F}' = (\mathcal{S}', \mathcal{R}')$, reconstruct PostgreSQL-compatible statistics:

- new MCV list: $\mathcal{M}'$
- new residual histogram bounds: $\mathcal{H}'$
- invariant: $\sum_i f'_i + p'_{\text{res}} = 1 - f_{\text{null}}$

**Algorithm 3** Decompose Canonical Full Distribution to PostgreSQL Statistics

Full distribution $\mathcal{F}' = (\mathcal{S}', \mathcal{R}')$, MCV threshold $\tau$, MCV limit $K$, target histogram size $m+1$ MCV list $\mathcal{M}'$, residual histogram bounds $\mathcal{H}'$ **Step 1: Extract MCV candidates** $\mathcal{C} \leftarrow \{(v, f) \mid (v, f) \in \mathcal{S}' \text{ and } f \geq \tau\}$ Sort $\mathcal{C}$ by frequency (descending) $\mathcal{M}' \leftarrow$ top $K$ entries from $\mathcal{C}$ **Step 2: Build residual distribution** $V_{\text{mcv}} \leftarrow \{v \mid (v, f) \in \mathcal{M}'\}$ Construct residual mixture from all range buckets in $\mathcal{R}'$ and all singletons in $\mathcal{S}'$ whose value is not in $V_{\text{mcv}}$ **Step 3: Resample equi-depth histogram bounds** $j = 0$ **to** $m$ Set $u_j$ to the residual quantile at level $j/m$ $\mathcal{H}' \leftarrow \{u_0, \ldots, u_m\}$ Validate: $\sum_{(v,f) \in \mathcal{M}'} f + p'_{\text{res}} = 1 - f_{\text{null}}$ $\mathcal{M}'$, $\mathcal{H}'$

### 4.8 MCV Selection Policy

The MCV threshold $\tau$ and limit $K$ control the final decomposition:

- **High threshold**: keeps the MCV list compact but pushes more probability mass into the residual histogram

– **Low threshold**: retains more explicit singleton masses as MCV entries
– **PostgreSQL default-like regime**: $\tau \approx 0.01$, $K = 100$

This threshold is a decomposition policy choice, not part of OASIS's core correction model. In our validation, experiments 1–3 remove threshold effects by using threshold-free round trips, while experiment 4 isolates the effect of $\tau_{\text{MCV}}$ after a simulated correction.

### 4.9 Why This Is a Portable Interface, Not a PostgreSQL Patch

The PostgreSQL instantiation is only one instance of the conversion pattern. The same interface idea extends to other engines that rely on histograms to very different degrees:

– **Self-contained histograms** (MySQL, MariaDB): direct pass-through; conversion degenerates to a near-identity mapping.
– **MCV-coupled statistics** (PostgreSQL, Oracle): reconstruct from explicit frequent-value summaries plus residual histograms, then decompose back.
– **Hybrid summaries** (SQL Server): reconstruct from histogram steps plus density summaries, apply OASIS, then recompute auxiliary summaries.

The systems contribution is therefore broader than "support PostgreSQL". The conversion interface turns histogram correction into a reusable component for heterogeneous optimizer statistics architectures.

### 4.10 Consistency Guarantees

### 4.11 Probability Mass Conservation

**Theorem 1 (Probability Conservation).** *If the source statistics satisfy the non-null invariant* $\sum_i f_i + p_{\text{res}} = 1 - f_{\text{null}}$*, then:*

1. *reconstruction produces a canonical full distribution with the same non-null mass;*
2. *decomposition emits engine-native statistics whose MCV mass plus residual mass again equals* $1 - f_{\text{null}}$*.*

*Proof.* Reconstruction assigns all explicit MCV mass to singleton buckets and all remaining non-null mass to residual intervals by construction. Decomposition selects an explicit MCV set and resamples the remainder as residual histogram bounds, so the output MCV mass and residual mass partition the same corrected non-null mass. Therefore the invariant is preserved.

### 4.12 Approximate Idempotence

**Definition 1 (Approximate Idempotence).** *Let* $(\mathcal{M}', \mathcal{H}') = \text{Decompose}(\text{Reconstruct}(\mathcal{M}, \mathcal{H}), \tau, K)$*. The conversion interface is* $\epsilon$*-idempotent if the round trip preserves total non-null mass within* $\epsilon$ *and preserves the residual quantile function within* $\epsilon$ *over a fixed evaluation grid.*

Perfect idempotence need not hold under thresholded decomposition because:

- the MCV threshold may change which values remain explicit,
- low-frequency singleton mass may be folded back into the residual distribution,
- floating-point rounding accumulates.

Our threshold-free validation shows exact total-mass preservation and machine-precision residual-quantile fidelity for the tested PostgreSQL-style statistics.

### 4.13 Implementation Interface

We provide a Python reference interface that exposes the conversion pattern explicitly:

Listing 1.1: Statistics-Format Conversion Interface (PostgreSQL Instance)

```python
class PostgreSQLStatisticsAdapter:
    def to_full_histogram(
        self,
        pg_stats: PostgreSQLStatistics
    ) -> FullHistogram:
        """Phase 1: Reconstruction"""
        pass

    def from_full_histogram(
        self,
        full_hist: FullHistogram
    ) -> PostgreSQLStatistics:
        """Phase 3: Decomposition"""
        pass

    def round_trip_test(
        self,
        pg_stats: PostgreSQLStatistics
    ) -> Tuple[bool, str]:
        """Validate round-trip fidelity"""
        pass
```

The full reference interface is available in `appendix/mcv_adapter_interface.py`. This appendix is the supplementary counterpart of the compact portability discussion in the main text and is intended to be kept numerically consistent with `experiments/results/mcv_validation_results.json`.

### 4.14 Performance Considerations

### 4.15 Time Complexity

- **Reconstruction**: $O(k + n)$ where $k = |\mathcal{M}|$ and $n = |\mathcal{H}|$
- **Decomposition**: dominated by MCV candidate sorting and residual quantile sampling
- **Total overhead**: negligible relative to correction and well within planning-time constraints in our PostgreSQL validation

### 4.16 Space Complexity

The canonical full distribution is temporary and small:

$$\text{Space}(\mathcal{F}) = \text{Space}(\mathcal{M}) + \text{Space}(\mathcal{H}) + O(1).$$

In practice this overhead is acceptable because statistics objects are compact, the canonical distribution exists only during correction, and no persistent storage expansion is required.

### 4.17 Validation Summary

We validate the PostgreSQL instantiation of the conversion interface on synthetic PostgreSQL-style statistics spanning uniform, normal, Zipfian, and bimodal distributions.

Table 1: PostgreSQL Statistics-Format Interface Validation Summary

| **Metric** | **Result** |
|---|---|
| Round-trip total mass error | 0 |
| Residual quantile MAE (mean) | $2.4 \times 10^{-17}$ |
| Residual quantile MAE (max) | $9.2 \times 10^{-17}$ |
| Selectivity MAE | $1.1 \times 10^{-4}$ |
| Selectivity P99 | $3.8 \times 10^{-3}$ |
| Default-like overhead (100 MCV, 10–20 bins) | 0.066–0.073 ms |
| Worst tested overhead (50 MCV, 50 bins) | 1.03 ms |

These results support three claims. First, the conversion interface is exact in mass preservation and essentially exact in residual-quantile reconstruction under threshold-free round trips. Second, it introduces negligible estimation bias and low latency on PostgreSQL-like settings. Third, threshold sensitivity is graceful: increasing $\tau_{\text{MCV}}$ shrinks the MCV list and gradually increases selectivity error, but does not compromise mass conservation.

### 4.18 Takeaway

The main value of this interface is not merely that it accommodates one PostgreSQL-specific edge case. Rather, it makes histogram correction portable to database systems whose optimizer statistics are stored as tightly coupled multi-component objects. OASIS corrects the canonical full-distribution view; the conversion interface is what makes that correction deployable across heterogeneous DBMS statistics architectures.

## 5 Predicate-Coverage Sensitivity Analysis

OASIS correction quality depends on the observation window covering the value range relevant to future predicates. We conduct a controlled sensitivity experiment

Table 2: Predicate-Coverage Sensitivity at $q$=10

| Coverage $w$ | Stale Prior Q-Error | OASIS Q-Error | Improvement |
|---|---|---|---|
| 1.00 (full) | 2.720 | **1.232** | 55% |
| 0.70 | 2.720 | **1.268** | 53% |
| 0.50 | 2.720 | **1.395** | 49% |
| 0.30 | 2.720 | **1.625** | 40% |
| 0.15 | 2.720 | **1.890** | 30% |

at $q$=10 by restricting observations to a "hot region" of width $w$ centered at the domain midpoint.

At $w$=0.7 (70% coverage), Q-Error degrades by only 3% relative to full coverage. At $w$=0.3, degradation reaches 40% but OASIS still outperforms the stale prior by 26%. Even at $w$=0.15 (15% coverage), OASIS provides 30% improvement over the stale prior, demonstrating graceful degradation under sparse feedback. The abstention policy (§ H in the main paper) provides a natural guard for extreme sparsity scenarios.

## 6 Observation Aggregation Study

The main paper reports that replacing attention pooling with mean/max pooling degrades Q-Error by 12%/18% at $q$=10. Here we provide the full breakdown.

Table 3: Observation Aggregation Ablation at $q$=10

| Pooling Strategy | Q-Error | Degradation vs. Attention |
|---|---|---|
| Attention (OASIS) | **1.232** | — |
| Mean pooling | 1.380 | +12% |
| Max pooling | 1.453 | +18% |

Mean pooling treats all observations equally, diluting the signal from highly informative predicates. Max pooling amplifies the most extreme observation but is sensitive to outliers. Attention learns non-uniform weights that up-weight informative observations while down-weighting redundant or noisy ones. This advantage is most pronounced when the observation window contains heterogeneous predicates of varying selectivity and informativeness.

## 7 Optimizer-Only EXPLAIN Protocol Details

The optimizer-only evaluation protocol uses PostgreSQL's `EXPLAIN` (without `ANALYZE`) to validate the statistics→plan causal link. `EXPLAIN` invokes the full optimizer pipeline—parsing, rewriting, planning, and cost estimation—but never executes the query, making it safe for repeated invocation on production systems.

*Protocol steps.*

1. Create a synthetic table with 200K rows and known distribution.
2. Run `ANALYZE` to capture fresh statistics.
3. Apply controlled DML mutations to create drift.
4. For each test query, run `EXPLAIN` under three statistics states: **(a)** stale (pre-mutation statistics), **(b)** fresh (post-mutation statistics captured by re-running `ANALYZE`), **(c)** OASIS-corrected (stale statistics with quantile boundaries replaced by OASIS predictions).
5. Compare plan structures: scan types, join methods, join order.

*Results.* On a TPC-DS-style scenario with extreme drift ($q \gg 10$):

- 8 queries tested, 2/8 (25%) showed scan-type transitions (Index Scan → Bitmap Heap Scan) when switching from stale to fresh statistics.
- OASIS-corrected statistics produced the same scan-type transitions as fresh statistics in both cases, confirming that the CBO's plan selection responds to the corrected quantile boundaries.
- Planning cost: $< 5$ ms per `EXPLAIN` invocation.

This protocol validates the causal chain statistics correction → selectivity change → cost re-estimation → plan change without requiring query execution, making it suitable for production-safe plan-impact validation.

## 8 Detailed Overhead Breakdown

Table 4: OASIS Runtime Overhead Breakdown (1000 trials)

| Component | p50 (ms) | p95 (ms) | p99 (ms) |
|---|---|---|---|
| Feature tensorization | 0.38 | 0.42 | 0.45 |
| Model forward pass | 0.66 | 0.72 | 0.78 |
| Total correction | 1.04 | 1.18 | 1.25 |
| Statistics-format conversion | 0.07 | 0.07 | 0.07 |
| Combined correction + conversion | 1.11 | 1.25 | 1.32 |
| Feedback collection (per tuple batch) | 0.12 | 0.18 | 0.23 |

For comparison, a column-level `ANALYZE` on a $10^7$-row table costs an estimated 300–500 ms of sequential I/O. OASIS's combined latency of ≈1.13 ms is ≈300× cheaper per invocation, well within the 200 ms planning budget (desideratum **D2**).

The statistics-format conversion interface adds negligible overhead (0.066–0.073 ms with PostgreSQL-like defaults of 100 MCV entries and 10–20 residual bins). Even in the worst tested configuration (50 MCV, 50 bins), the conversion completes in 1.03 ms.

## References

1. Bruno, N., Chaudhuri, S., Gravano, L.: STHoles: A multidimensional workload-aware histogram. In: Proceedings of SIGMOD. pp. 211–222 (2001)
2. Markl, V., Megiddo, N., Kutsch, M., Tran, T.M., Lang, P., Raman, V.: Consistent selectivity estimation via maximum entropy. The VLDB Journal **16**(2), 169–188 (2007)
3. Park, Y., Zhong, S., Mozafari, B.: QuickSel: Quick selectivity learning with mixture models. In: Proceedings of SIGMOD. pp. 1017–1033 (2020)
4. Rathuwadu, L., Liu, G., Leckie, C., Borovica-Gajic, R.: Colse: A lightweight and robust hybrid learned model for single-table cardinality estimation using joint cdf. arXiv preprint arXiv:2512.12624 (2025). `https://doi.org/10.48550/arXiv.2512.12624`